



Week 7 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@October 18, 2021 2:13 PM
🔗 Materials	
# Module #	39
▼ Type	Lecture
# Week #	7

Data Search

O() notation - The Big O notation

- Used in study of algorithmic complexity
- Rough approximation: "order of magnitude", "approximately", etc.
- Main concepts:
 - $O(1)$ - constant time, independent of input size: Excellent!
 - $O(\log N)$ - logarithmic in input size - grows very slowly with input: Very good
 - $O(N)$ - linear in input size - often the baseline - Would like to do better
 - $O(N^k)$ - polynomial (quadratic, cubic, etc.) - Not good as the input size grows
 - $O(k^N)$ - exponential - **VERY BAD**: Won't even work for reasonably small inputs

Searching for an element in the memory

Unsorted data in a linked list (or an array)

- Start from the beginning
- Proceed stepwise, comparing each element
- Stop if found and return the LOCATION
- If end-of-list, return NOTFOUND

$O(N)$

Sorted data in an array

- Look at the middle element in an array:
 - Greater than the target?
 - Search in the lower half
 - Lesser than the target?
 - Search in the upper half
- Switch focus to the new array: Half the original size
 - Repeat

$O(\log N)$: *Binary Search*

Problems with arrays

- Size must be fixed ahead of time
- Adding new entries requires re-sizing - can try oversize, but eventually ...
- Maintaining sorted order $O(N)$:
 - Find the location to insert
 - Move all further elements by 1 to create a gap
 - Insert
- Deleting
 - Find the location, delete
 - Move all the entries down by 1 step

Alternatives

- Binary Search Tree
 - Maintaining sorted order is easier: growth of the tree
- Self-Balancing
 - BST can easily tilt to one side and grow downwards
 - Red-black, AVL, B-tree ... more complex, but still reasonable
- Hash tables
 - Compute an index for an element: $O(1)$
 - Hope the index for each element is unique!
 - Difficult but is doable in many cases